

■ Sample Slide

■ Solution Architecture Proposal

Solution Architect :Phyo Thu Han

Executive Summary

2

- Provides a wide range of services, including **search engine ,record searching and search with images.**
- Following services are included in system.

Search Service

Enable image, video, and text search with fast response using Redis caching and RabbitMQ async processing

(Example: Customer Screening

Customer uploads PEP data (CSV/Excel), and the system processes and indexes the data, making it available for search and external system API integration for screening.)

Search Data Service

Process and index search data efficiently using queue-based ingestion and batch processing

Browser Caching

Improve user experience and reduce backend load using browser caching and CDN

Objectives

3

Provide accurate search results within acceptable response time while supporting multiple search formats and capturing search data for future performance optimization.

Design Principles

4

Principle	Description
Simplicity	Use a .NET modular microservice architecture to keep the system straightforward, scalable, and easy to maintain.
Clean Code Architecture & SOLID Principle	Develop each service using Clean Architecture, modular design, and SOLID principles to improve maintainability, testability, and future enhancement.
Security First	Use JWT/OAuth2 for authentication and authorization. Apply RBAC for business functions and sensitive data access. Implement API Gateway security controls including rate limiting, request validation, IP allowlist, and centralized authentication.
Audit trail	Log all user and system actions at detailed action level for traceability, compliance, and issue investigation.

Propose Scope

Search Service

- Handle user search requests (image, video, text) with fast response using Redis caching and RabbitMQ async processing
- Retrieve cached results from Redis and query search index for real-time data
- Publish search activity to RabbitMQ for logging and analytics processing
- Return ranked search results and cache frequent queries for performance improvement

Search Data Service

- Ingest raw data and publish to RabbitMQ for asynchronous indexing
- Process and index data into search engine database for fast retrieval
- Update Redis cache with frequently accessed search data
- Execute batch indexing to handle large-scale data updates efficiently

Propose Scope

6

Browser Caching

- Cache search responses using browser caching (Cache-Control, ETag)
- Reduce repeated backend requests by serving cached results
- Use CDN to deliver content closer to users
- Validate cached data before making backend calls

Solution Boundaries

7

- Indexing and archival apply only to selected datasets; historical data may require manual query access
- Retry mechanism applies only to search processing and data ingestion (indexing pipeline)
- External data source failures/timeouts (e.g., third-party content providers) are outside system control
- Data synchronization is near real-time (not real-time) due to end of data upload
- Incomplete or failed data ingestion may require manual reprocessing or re-upload
- Browser caching behavior depends on client configuration and cannot be fully controlled by backend
- No infrastructure changes (e.g., hardware upgrades, CDN expansion) are included in this scope
- Advanced features (e.g., AI ranking, recommendation engine) are excluded from this phase

Assumptions

Search Platform

Search data sources are valid and consistently available for indexing and querying
User search patterns are predictable and suitable for caching and optimization
Data volume is within manageable limits for indexing and retrieval performance

Data Synchronization

External systems provide accessible APIs or data feeds for synchronization
Data synchronization frequency supports near real-time (not real-time) updates
Batch processing is sufficient to handle incoming data volume

Example

Data Integration

- Upload PEP data via CSV/Excel → system processes and indexes for search
- Configure rules via form or Excel for screening logic
- Upload T+1 data to file server for batch ingestion

Customer screening

- Customer data received → system checks blacklist and transaction patterns → returns screening result

Assumptions

Caching⁹ & Messaging

Redis is available for high-frequency data caching and low-latency access

RabbitMQ is stable and supports asynchronous processing for search and indexing flows

Queue processing latency is within acceptable limits for near real-time behavior

Client & Performance

Client applications (browser/mobile) support caching mechanisms (Cache-Control, ETag)

Network latency between services is within acceptable range

Monitoring and logging mechanisms are available for system observability

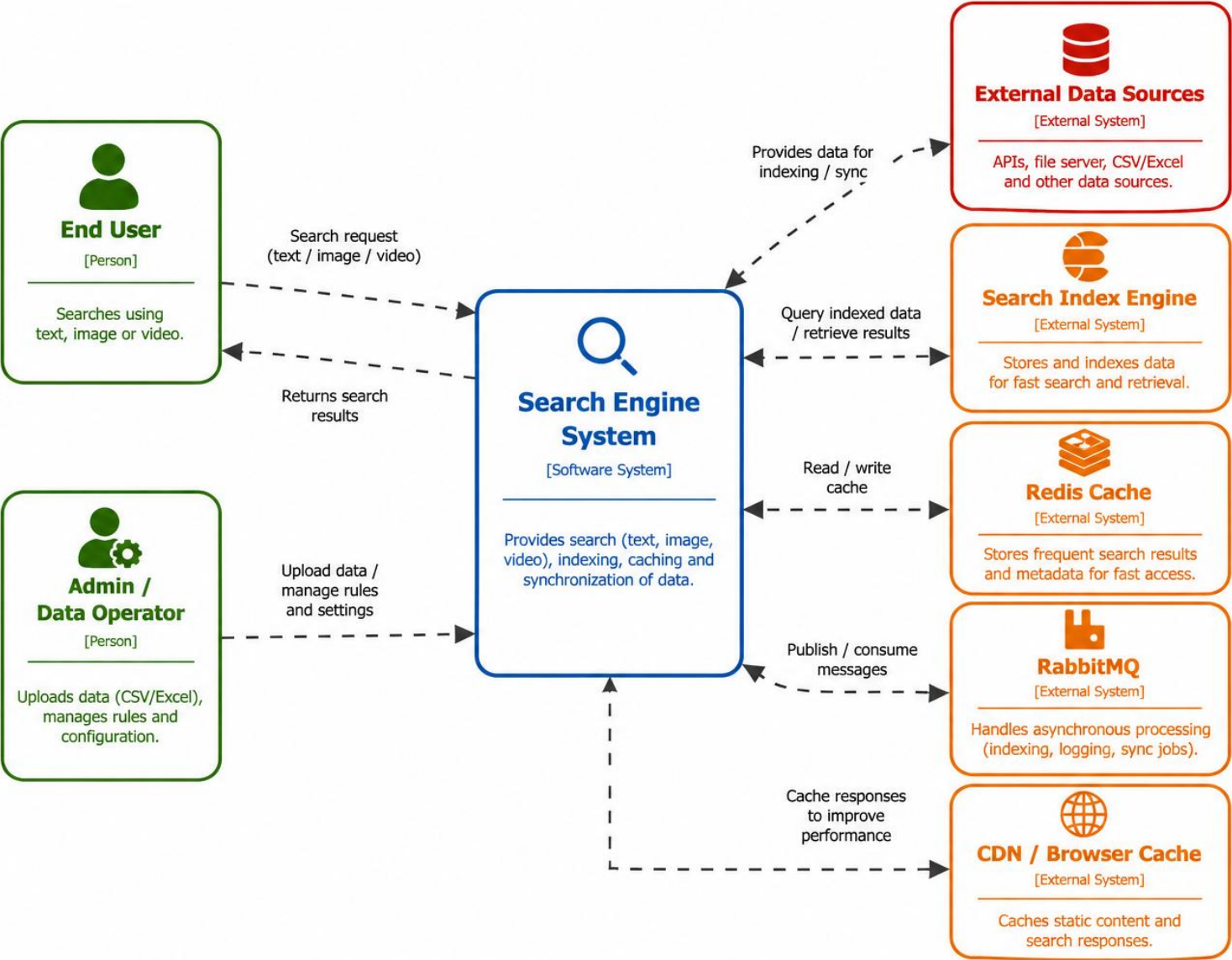
Constraints

10

- Eventual consistency model → temporary mismatch between source data and search index is possible
- Retry mechanism → risk of duplicate indexing or duplicate processing without idempotency control
- Redis dependency → cache failure may impact search response time
- Search index dependency → index failure may affect search accuracy and result availability
- External data source dependency → limited control over third-party API/feed failures
- Browser caching limitation → cached results may be outdated until cache refresh or invalidation
- Increased system complexity → requires monitoring, alerting, and operational readiness
- Near real-time synchronization → search results may not reflect latest data immediately

In diagram , although this sample diagram is acceptable(not 100% correct due to unclear scenario) but I don't like much but I add as sample .what I prefer is only following C4 url diagram

<https://c4model.com/diagrams/system-context>

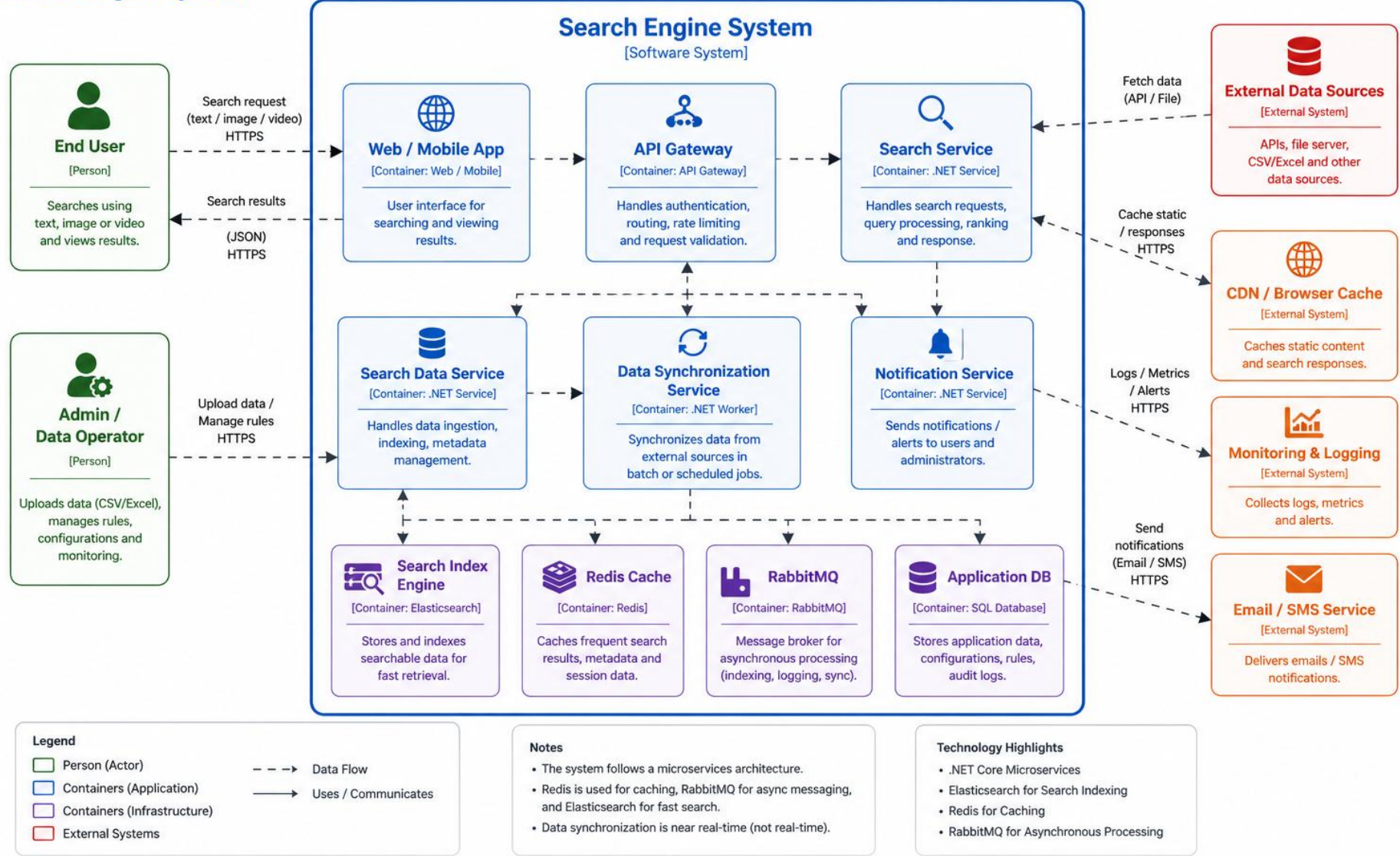


System Context View: Search Engine System

The system context diagram for a Search Engine System

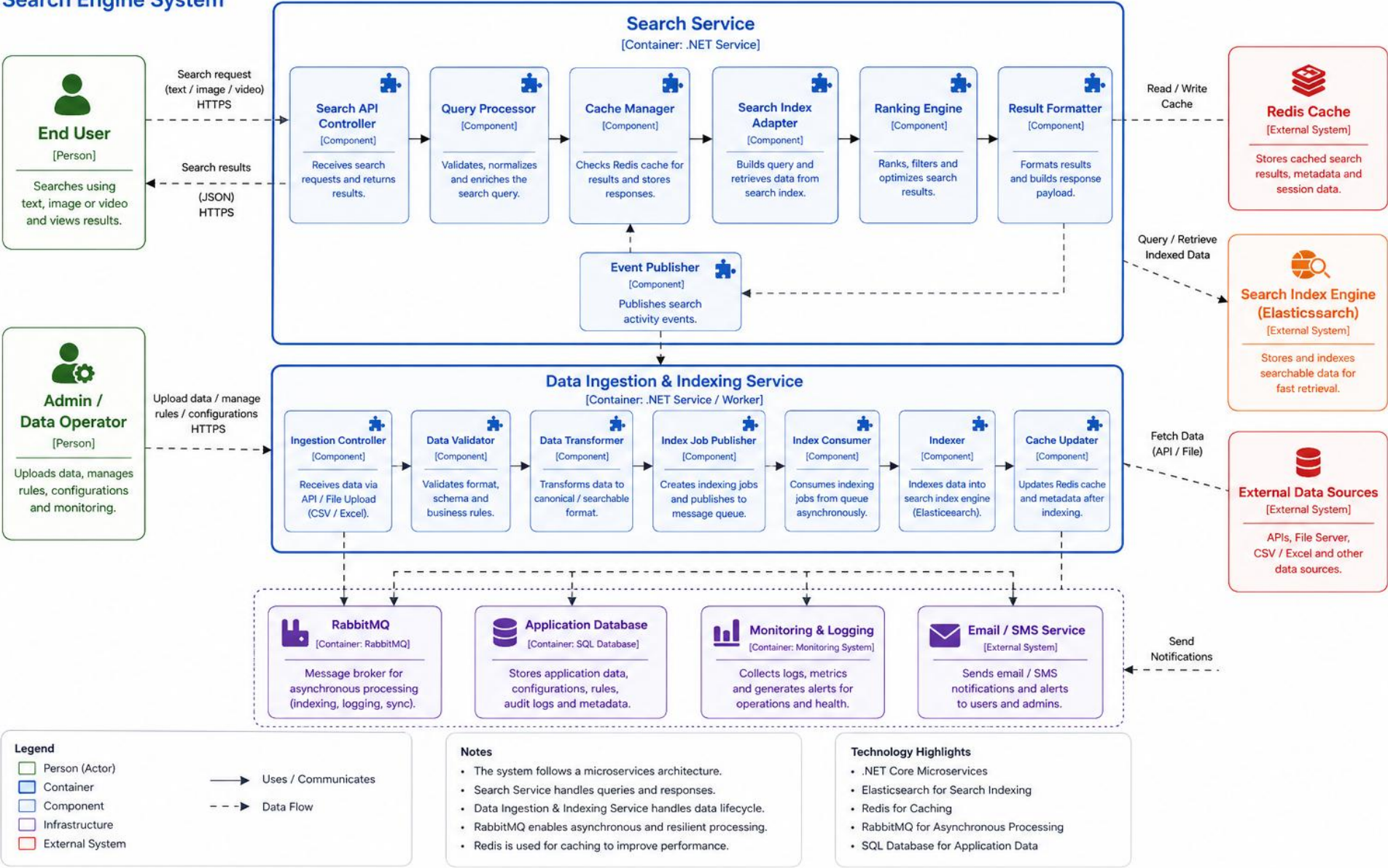
High Level Architecture (C4 Container Diagram)

C4 Level 2 – Container Diagram Search Engine System



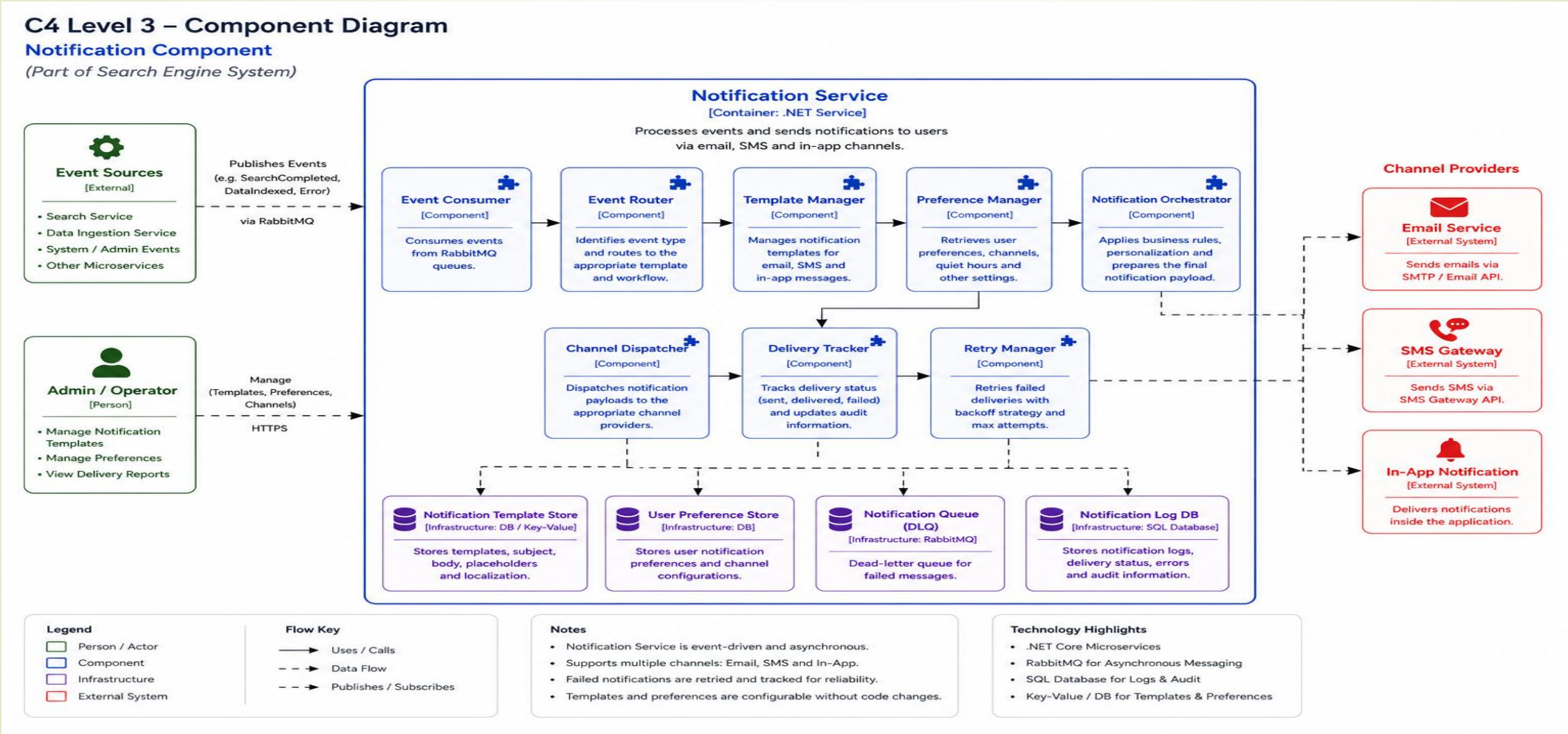
Key Components (C4 Component Diagram for Search Service)

C4 Level 3 – Component Diagram Search Engine System



Key Components (Component Diagram for Notification Service)

you need to add more than one diagram if you need to explore more detail for each component

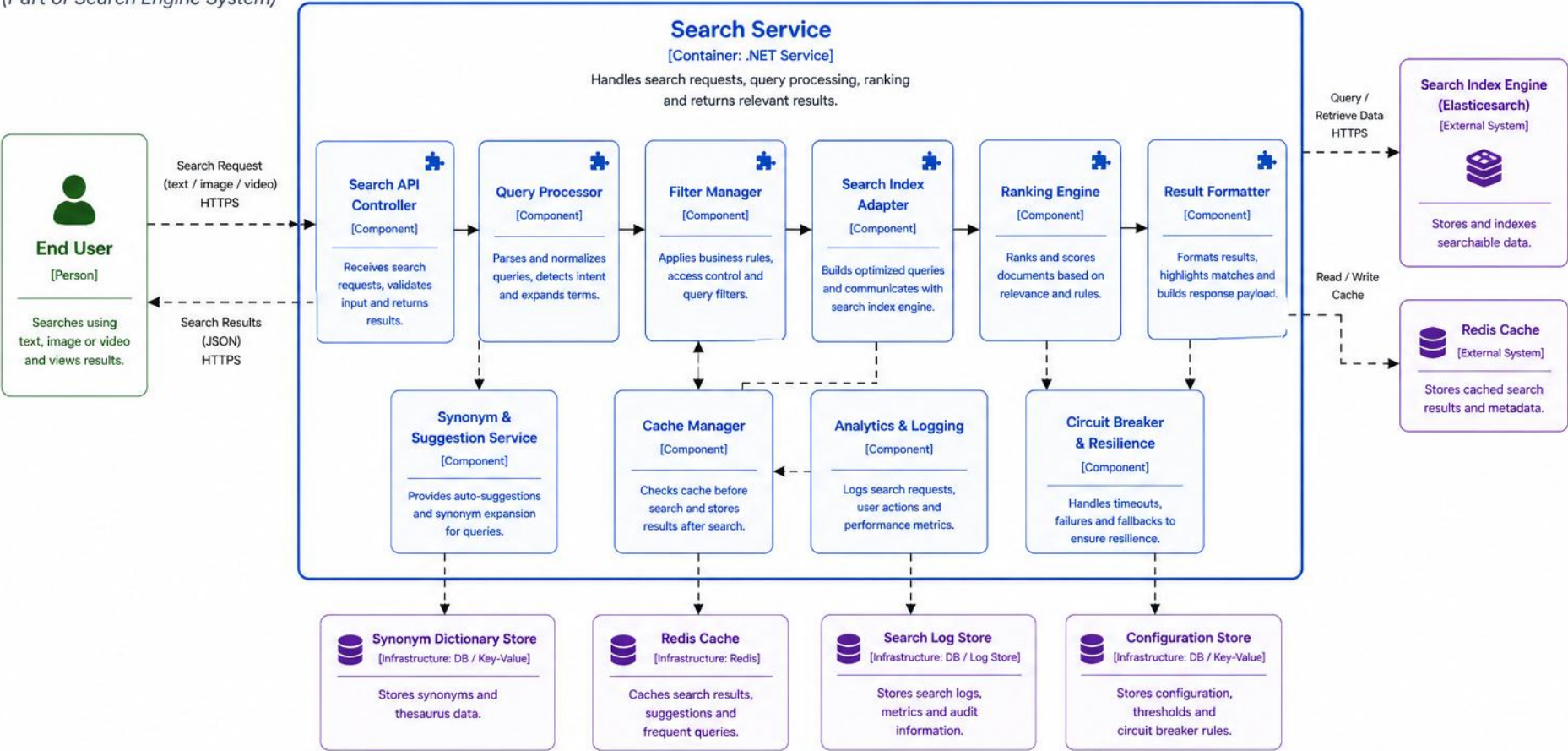


Key Components (Component Diagram for Search Service)

C4 Level 3 – Component Diagram

Search Service Component

(Part of Search Engine System)



Legend

- Person (Actor)
- Component
- Infrastructure
- External System

Flow Key

- Uses / Calls
- Data Flow
- Read / Write

Notes

- Search Service is stateless and scalable.
- Caching improves performance and reduces index load.
- Resilience patterns ensure high availability.
- Analytics help in monitoring and improving search quality.

Technology Highlights

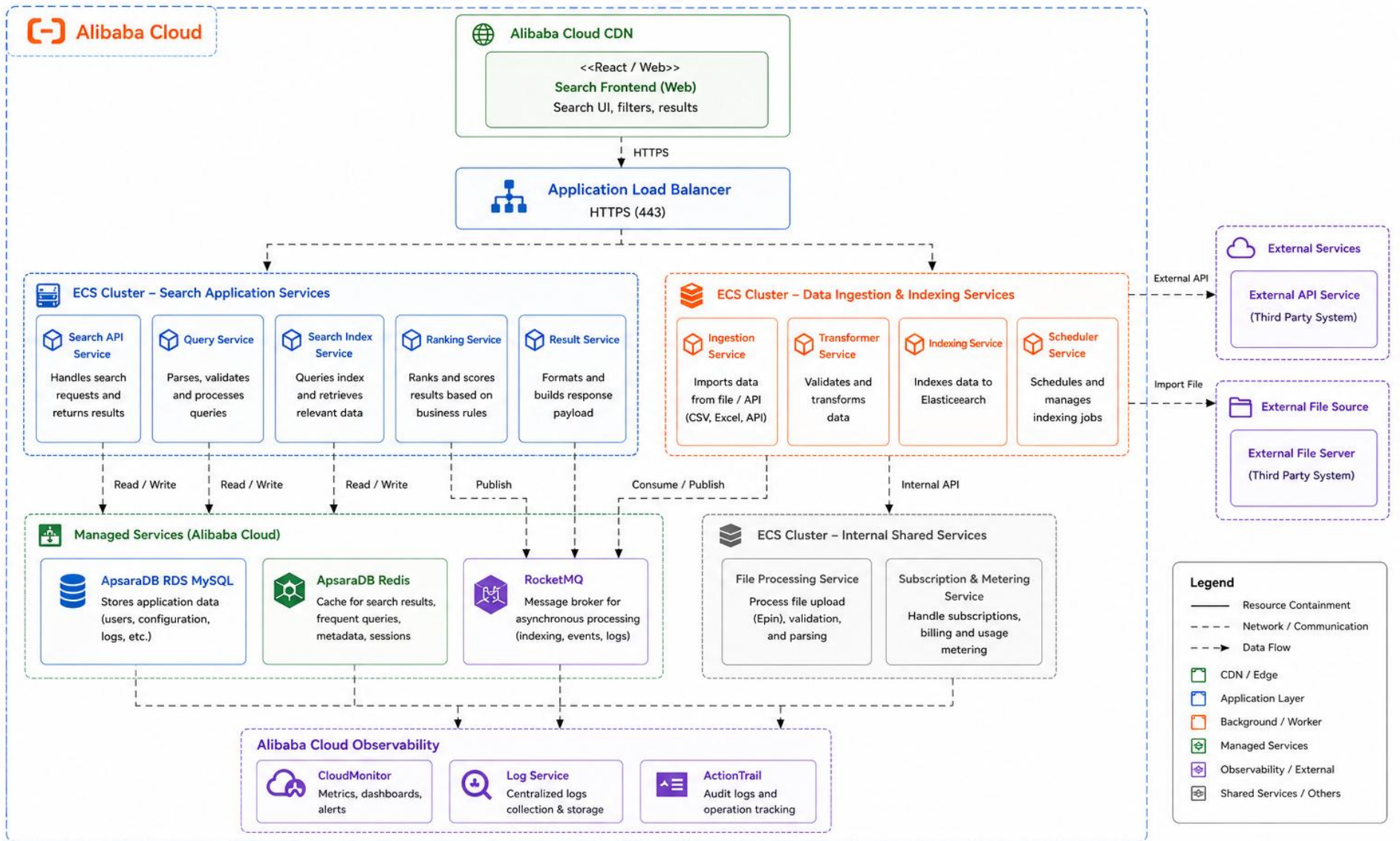
- .NET Core Microservices
- Elasticsearch for Search Indexing
- Redis for Caching
- Structured Logging & Monitoring

Swimlane Example(Add more than one diagram base on process which you high light screening diagram , report diagram , alert diagram)



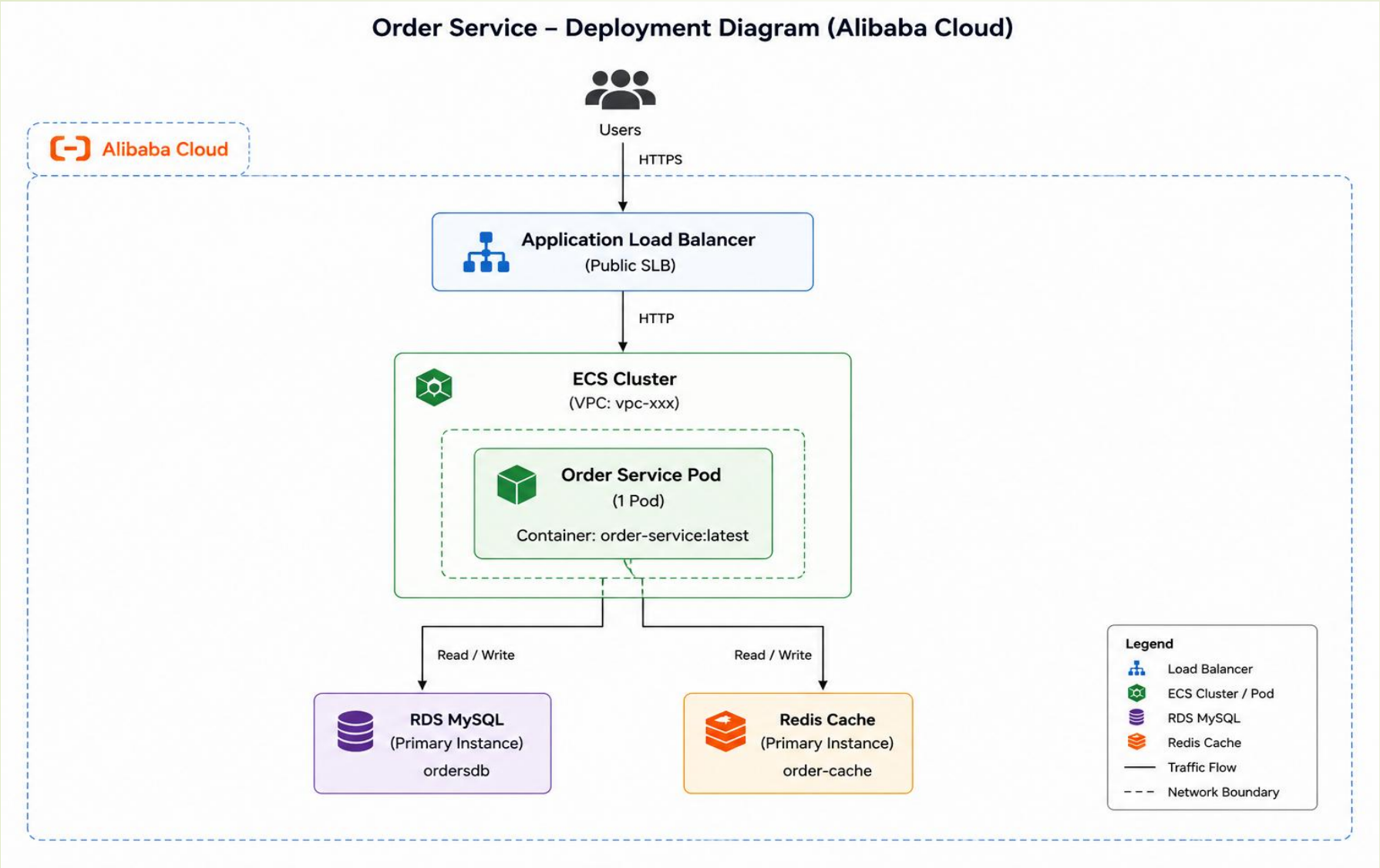
Deployment Diagram

Search Engine System – Deployment Diagram (Alibaba Cloud)



Note: This deployment diagram represents only the Search Engine System hosted on Alibaba Cloud. All external systems (APIs, file sources, third party services) are outside the scope and hosted outside Alibaba Cloud.

Deployment Diagram (Another Example for another System not search engine simple deployment)



Tech Stack

20

Layer	Component	Technology/Tools	IDE
Frontend		React Native	VSCode
Application Platform Infra	Backend Service	Java(Spring Boot)	Intellij, VSCode
	Runtime	Ali Cloud Kubernetes Cluster	
Data Storage	Backend DB	ApsaraDB For MySQL	
	Message Broker	Rabbit MQ	
	Cache Management	Redis Cache	
Integration Service	API Gateway	WSO2	
	Interfacing System	REST	
Documentation	API docs	Swagger	

BOM

Non-Prod 1

No.	Description	vCPU(Cores)	Memory(GB)	Storage(GB)	Qty	Remark	
1.	ACK(Kubernetes Cluster)	4	16	200	1	3 nodes cluster	
2.	ALB(Kubernetes ingress)				1		
3.	Pod	0.5	0.256	200	4		
4.	Cloud Backup			200	1		
7.	Redis	1	1	10 GB	1		
8.	Rabbit MQ	1	2	10 GB	1		

No.	Description	vCPU (Cores)	Memory(GB)	Storage (GB)	Qty	Remark
1.	ACK(Kubernetes Cluster)	4	16	200	1	3 nodes cluster
2.	Pod	0.5	0.256	200	4	
3.	ALB (Kubernetes ingress)				1	
6.	Redis	1	1	10 GB	1	
7.	Cloud Backup			200	1	
8.	Rabbit MQ	2	4	20 GB	1	

Required APIs For External Api

00

No	API Name	Description
1	check/AML	To top up mobile number

Thank You!